Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

По дисциплине: Информационная безопасность

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

Работа 1: Разработка защищенного REST API с интеграцией в CI/CD

Группа: P3414
Сделал: Козьяков Арсений Дмитриевич
Проверил: Маркина Татьяна Анатольевна

Санкт-Петербург
2026

Содержание

1 Работа 1: Разработка защищенного REST API с интеграцией в CI/CD **2**
1.1 Описание проекта 2
1.1.1 Используемый стек 2
1.1.2 Архитектура 2
1.2 Описание API 2
1.2.1 Общее описание 2
1.2.2 Поддерживаемые методы 2
1.3 Описание реализованных мер защиты 4
1.3.1 SQLi 4
1.3.2 XXS 4
1.3.3 Реализация аутентификации 4
1.4 Тесты 5
1.5 Pipeline 10

1 Работа 1: Разработка защищенного REST API с интеграцией в CI/CD

1.1 Описание проекта

1.1.1 Используемый стек

- Java 21
- SpringBoot 4.1.1v
- maven

1.1.2 Архитектура

- /src - код SpringBoot приложения
- /.github/ - настроенный CI
- /report-template - шаблон для генерации отчета
- [/.env.example] - пример для переменных окружения
- [/compose.yaml] - файл для создания docker контейнера с PostgreSQL
- [Makefile] - Makefile для удобства генерации отчета (make pdf)

1.2 Описание API

1.2.1 Общее описание

Система предназначена для управления студентами, поля:

- Имя (строка)
- Фамилия (строка)
- Отчество (строка)
- Группа (строка)

Базовый адрес API:

http://localhost:8080

Данные передаются в формате JSON. Для защищённых методов используется HTTP-заголовок:

Authorization: Bearer

1.2.2 Поддерживаемые методы

1.2.2.1 POST /auth/login Авторизация пользователя и получение JWT-токена

Тело запроса:

```
{
  "username": "admin",
  "password": "password123"
}
```

Успешный ответ (200 OK):

```
{
  "accessToken": "<JWT>",
  "tokenType": "Bearer",
  "expiresIn": 3600
}
```

Ошибка (401 Unauthorized): Неверный логин или пароль

1.2.2.2 GET /api/data Возвращает список всех студентов. Доступ предоставляется только аутентифицированным пользователям.

Пример запроса:

```
curl -i http://localhost:8080/api/data \
  -H "Authorization: Bearer <JWT>"
```

Успешный ответ (200 OK):

```
[
  {
    "id": 1,
    "firstName": "Иван",
    "lastName": "Иванов",
    "patronymic": "Иванович",
    "group": "P3414"
  }
]
```

1.2.2.3 POST /api/data Тело запроса:

```
{
  "firstName": "Петр",
  "lastName": "Петров",
  "patronymic": "Петрович",
  "group": "P3414"
}
```

Пример запроса:

```
curl -i -X POST http://localhost:8080/api/data \
         -H "Authorization: Bearer <JWT>" \
         -H "Content-Type: application/json" \
         -d '{
             "firstName": "Петр",
                 "lastName": "Петров",
                 "patronymic": "Петрович",
                 "group": "P3414"
         }'
```

Успешный ответ (201 Created): Возвращает созданный объект студента с присвоенным id

```
{
  "id": 2,
  "firstName": "Петр",
  "lastName": "Петров",
  "patronymic": "Петрович",
  "group": "P3414"
}
```

При некорректных данных возвращается 400 Bad Request, а при отсутствии действительного JWT — 401 Unauthorized.

1.3 Описание реализованных мер защиты

1.3.1 SQLi

Для снижения риска SQL-инъекций в приложении используется Spring Data JPA с Hibernate. Доступ к данным выполняется через репозитории, а SQL-запросы не формируются конкатенацией пользовательских строк.

1.3.2 XXS

В приложении пользовательские поля студента проходят HTML-экранирование при формировании DTO ответа с помощью HtmlUtils.htmlEscape.

Например, строка:

```
<script>alert(1)</script>
```

преобразуется в безопасное текстовое представление:

```
&lt;script&gt;alert(1)&lt;/script&gt;
```

1.3.3 Реализация аутентификации

Для защиты API от несанкционированного доступа используется Spring Security и механизм JWT.

При успешной аутентификации сервер формирует JWT с использованием алгоритма HS256. Токен подписывается секретным ключом, заданным через переменные окружения.

В токен включаются основные утверждения:

sub — имя пользователя;

iss — издатель токена;

iat — время выдачи;

exp — время истечения срока действия.

При обращении к защищённым эндпоинтам Spring Security проверяет подпись и срок действия токена, а также ожидаемого издателя.

1.4 Тесты

0) Подготовка

```
set -a
source .env
set +a

mvn: clean spring-boot:run
```

1) Успешная аутентификация

```
curl -i -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d "$(jq -n \
    --arg username "$APP_BOOTSTRAP_USERNAME" \
    --arg password "$APP_BOOTSTRAP_PASSWORD" \
    '{username: $username, password: $password}')"
```

Ответ:

```
HTTP/1.1 200
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 19 Oct 2026 07:17:02 GMT

{"accessToken":"eyJhbGciOiJIUz....","tokenType":"Bearer","expiresIn":3600}
```

Примечание: в ответе скращен accessToken, поскольку он слишком большой и мешает парситься другим полям, вот полное значение:

```
eyJhbGciOiJIUzI1NiJ9.eyJpc3
MiOiJzdHVkZW50LWFw
aSIsInN1YiI6
ImFkbWluIiwiZ
XhwIjoxNzkyMzk3ODIyLCJpY
XQiOjE3OTIzOTQyMjJ
9.W2UVNphxv0sCFQD9rxR9q4mv8okGH9TyZZH-btH_MJQ
```

2) Отказ при неправильном пароле:

```
curl -i -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d "$(jq -n \
    --arg username "$APP_BOOTSTRAP_USERNAME" \
    --arg password 'wrong-password-123' \
    '{username: $username, password: $password}')"
```

Ответ:

```
HTTP/1.1 401
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 19 Oct 2026 07:17:51 GMT

{"error":"Invalid username or password"}
```

3) Подготовка токена для последующих запросов:

```
TOKEN=$(curl -sS -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d "$(jq -n \
    --arg username "$APP_BOOTSTRAP_USERNAME" \
    --arg password "$APP_BOOTSTRAP_PASSWORD" \
    '{username: $username, password: $password}')" \
  | jq -r '.accessToken // empty')

test -n "$TOKEN" && echo "JWT получен" || echo "Ошибка получения JWT"
```

4) Защита GET от неавторизованного доступа

```
curl -i http://localhost:8080/api/data
```

Ответ:

```
HTTP/1.1 401
WWW-Authenticate: Bearer resource_metadata="http://localhost:8080/.well-known/o
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Mon, 19 Oct 2026 07:19:43 GMT
```

5) Получение списка с JWT

```
curl -i http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN"
```

Ответ:

```
HTTP/1.1 200
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
```

```
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Content-Length: 2
Date: Mon, 19 Oct 2026 07:21:13 GMT
```

6) Добавление стунедта с JWT

```
curl -i -X POST http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "firstName": "Петр",
    "lastName": "Петров",
    "patronymic": "Петрович",
    "group": "P3414"
  }'
```

Ответ:

```
HTTP/1.1 201
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 19 Oct 2026 07:22:06 GMT

{"id":1,"firstName":"Петр","lastName":"Петров","patronymic":"Петрович","group":
```

7) Запрет добавления без JWT

```
curl -i -X POST http://localhost:8080/api/data \
  -H 'Content-Type: application/json' \
  -d '{
    "firstName": "Unauthorized",
    "lastName": "User",
    "group": "TEST"
  }'
```

Ответ:

```
HTTP/1.1 401
WWW-Authenticate: Bearer resource_metadata="http://localhost:8080/.well-known/o
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
```

```
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Mon, 19 Oct 2026 07:23:05 GMT
```

8) Защита от SQL-инъекции в данных студента

```
curl -i -X POST http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d "$(jq -n \
    --arg firstName 'Robert' \
    --arg lastName "Robert'); DROP TABLE students;--" \
    --arg group 'P3414' \
    '{
      firstName: $firstName,
      lastName: $lastName,
      patronymic: null,
      group: $group
    }')"
```

```
HTTP/1.1 201
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 19 Oct 2026 07:25:37 GMT

{"id":2,"firstName":"Robert","lastName":"Robert&amp;#39;); DROP TABLE students;--",
```

8.1) Проверка доступности таблицы

```
curl -i http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN"
```

Ответ:

```
HTTP/1.1 200
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Content-Length: 227
Date: Mon, 19 Oct 2026 07:26:27 GMT
```

```
[{"id":2,"firstName":"Robert","lastName":"Robert&#39;); DROP TABLE students;--"
```

9) Защита от XSS

```
curl -i -X POST http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d "$(jq -n \
    --arg firstName '<script>alert(1)</script>' \
    --arg lastName 'XSS-Test' \
    --arg group 'XSS-TEST' \
    '{
      firstName: $firstName,
      lastName: $lastName,
      patronymic: null,
      group: $group
    }')"
```

Ответ:

```
HTTP/1.1 201
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 19 Oct 2026 07:30:07 GMT

{"id":3,"firstName":"&lt;script&gt;alert(1)&lt;/script&gt;","lastName":"XSS-Tes
```

10) Проверка валидации данных

```
curl -i -X POST http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "lastName": "Test",
    "group": "INVALID-TEST"
  }'
```

Ответ:

```
HTTP/1.1 400
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
```

```
Transfer-Encoding: chunked
Date: Mon, 19 Oct 2026 07:31:30 GMT
Connection: close

{"timestamp":"2026-10-19T07:31:30.540Z","status":400,"error":"Bad Request","pat
```

11) Отклонение подменённого JWT

```
TAMPERED_TOKEN="${TOKEN%.*}.invalid-signature"

curl -i http://localhost:8080/api/data \
  -H "Authorization: Bearer $TAMPERED_TOKEN"
```

Ответ:

```
HTTP/1.1 401
WWW-Authenticate: Bearer error="invalid_token", error_description="An error occ
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Mon, 19 Oct 2026 07:35:27 GMT
```

Все тесты прошли успешно

1.5 Pipeline

ссылка на последний успешный pipeline